Trip Planner Agent - Revised System Design

Sep 24, 2026 · @gopi

What Changed and Why

The original design correctly identifies the major building blocks (frontend, agent engine, tool connectors) but treats several critical decisions as implicit or deferred. This revision makes the following explicit:

- **Trip state is split from conversation memory** — the itinerary needs to be a durable, structured object, not something re-derived from chat history each turn.
- **Tool rollout is phased** — Places and Weather are cheap/free and ready now; Flights and Hotels require partner approval and paid access, so they are deferred to Phase 2.
- **Failure modes are designed for**, not left to happen at demo time.
- **Cost and latency guardrails** are specified upfront (max tool iterations, streaming strategy for tool calls).
- **Auth and persistence** are resolved as a decision rather than left open, since it changes the backend shape significantly.
- **Stack choice is committed**: Vercel AI SDK, not "LangChain or Vercel AI SDK."

Revised Architecture

The main additions are a **Trip State Store** separate from conversation memory, and **MCP (Model Context Protocol)** as the default way tools connect to external data, used wherever a well-maintained MCP server already exists for that data source — otherwise a direct API call, as a plain AI SDK tool.

```
flowchart TD
    User([User]) -->|Chat UI| Frontend

    subgraph NextJS["Next.js Application"]
        Frontend["Frontend: React UI"]
        API["Backend: Route Handlers"]
        Frontend <-->|REST / SSE| API
    end

    subgraph Agent["Agentic Backend (Vercel AI SDK)"]
```

```
        Engine["Agent Loop (ReAct)"]
        LLM["LLM Provider"]
        ConvoMem["Conversation Memory"]
        TripState["Trip State Store<br/>(Postgres/Supabase)"]
        MCPClient["AI SDK MCP Client"]
        Engine <--> LLM
        Engine <--> ConvoMem
        Engine <--> TripState
        Engine <--> MCPClient
    end

    subgraph MCPServers["MCP Servers (Phase 1 + 2)"]
        PlacesMCP["Google Maps MCP<br/>places + directions"]
        AmadeusMCP["Amadeus MCP<br/>flights + hotels"]
    end

    subgraph Direct["Direct API (no MCP available)"]
        Weather["Weather API"]
    end

    API <--> Engine
    MCPClient <--> PlacesMCP
    MCPClient -.-> AmadeusMCP
    Engine --> Weather
```

Key differences from the original diagram: trip state is a persisted first-class component; an MCP Client sits alongside direct tool calls, connecting to third-party MCP servers (Google Maps, Amadeus) where they exist, so the agent isn't maintaining its own API wrapper for every data source; Weather stays a direct API call since no MCP server for it is warranted yet.

Trip State Model

The agent's final synthesis step should target a strict schema (via the AI SDK's `generateObject` or typed tool args) rather than freeform text parsed after the fact. Proposed shape:

```
Trip {
  id: string
  userId: string | null        // null for anonymous/session-only trips
  destination: string
```

```
  startDate: date
  endDate: date
  budget: number | null
  transportMode: "walking" | "transit" | "driving" | "cycling" | "rideshare"
| null
  status: "draft" | "confirmed"
  days: Day[]
}

Day {
  date: date
  stops: Stop[]
}

Stop {
  time: string | null
  name: string
  type: "attraction" | "meal" | "flight" | "hotel" | "transport" | "other"
  location: { lat: number, lng: number } | null
  notes: string | null
  sourceToolCallId: string | null   // traceability back to the tool call
that produced this
}
```

The Itinerary View component renders directly off this object, not off parsed chat text. Each tool call that contributes a stop stamps `sourceToolCallId`, so a later failed/retried tool call can be reconciled without corrupting the rest of the itinerary.

Ground Transportation

Added per request: the agent should not assume a transport mode. During intake (alongside dates, budget, destination), the agent asks the user's preferred mode — walking, transit, driving, cycling, or rideshare — and also proactively suggests one based on destination and stop spacing (e.g. a dense old-town center suggests walking/transit over driving).

- **Ask**: added as a standard intake question, same turn as dates/budget, not a separate follow-up.
- **Suggest**: once stops are known, the agent checks distances between consecutive stops (via the Google Maps MCP server's directions/distance-matrix tools) and flags if the chosen mode looks impractical for a given day (e.g. a 12 km gap on a "walking" trip),

offering to switch that day's mode rather than silently leaving it inconsistent.

- **Where it lives**: `transportMode` on the `Trip` object is the trip-level default; a `Stop` of `type: "transport"` can override it for a specific leg (e.g. one intercity train on an otherwise walking-based trip).
- **Tool**: Google Maps MCP server's `maps_directions` / `maps_distance_matrix` tools, which natively support driving/walking/transit/cycling — rideshare price/booking would need the separate Uber MCP server, scoped as Phase 2 and optional (see Phased Tool Rollout).

Phased Tool Rollout

Phase	Tool	Access via MCP?	Notes
1	Places + Directions	Yes — Google Maps MCP server	Covers attraction lookup and local transport mode/route suggestions
1	Weather	No published MCP server worth adopting yet	Direct API call (OpenWeather, free tier)
2	Flights	Yes — community Amadeus MCP servers exist	Still needs an Amadeus developer account; MCP saves writing the wrapper, not the account setup
2	Hotels	Yes — same Amadeus MCP servers cover hotel search	Booking flow (vs. search) still needs partner approval
2	Rideshare (optional)	Yes — Uber MCP server exists	Only if in-app ride booking (not just mode suggestion) is in scope

MCP is the default integration path for Phase 1 and 2 tools where a server already exists — it means adopting someone else's maintained wrapper instead of writing and maintaining our own. It does not remove the underlying work for Phase 2 (Amadeus developer account, partner terms for actual bookings); it only removes the API-wrapper code. Where no server exists (Weather today), a direct AI SDK tool is still the right call — MCP is not used as an end in itself.

Error Handling and Tool Failure UX

- **Tool API failure (5xx, timeout, rate limit)**: agent catches the error, does not fabricate a substitute result, and either retries once with backoff or surfaces a specific gap to the user ("couldn't get weather for Paris right now, continuing without it") rather than silently omitting it.
- **No results for a valid query**: treated as a distinct case from a failure — the agent says so explicitly rather than inventing plausible-sounding attractions.
- **Malformed/missing tool call args from the LLM**: validated against the tool's schema before execution; a validation failure triggers a re-ask to the LLM with the specific error, not a raw exception to the user.
- **Partial itinerary on failure**: since trip state is a persisted object (see Trip State Model), a failed tool call for Day 2 should not block or corrupt Day 1 — failures are scoped per stop/day, not per whole-trip generation.

Cost, Latency, and Iteration Controls

- **Max tool-call iterations per turn**: cap the ReAct loop (e.g. 5 iterations) to guard against runaway tool-calling loops; on hitting the cap, synthesize with whatever data is available and note what's missing.
- **Streaming strategy**: show tool calls as they happen ("Checking flights to Paris...") rather than hiding the agent behind a single spinner until final synthesis — this is a perceived-latency win, not just a cosmetic one, since multi-tool turns can take several seconds.
- **Caching**: cache Places/Weather responses per destination for a short TTL (e.g. 1 hour) to cut redundant calls when a user iterates on the same trip.
- **Per-turn budget**: a soft ceiling on LLM calls + tool calls per user turn, logged, so cost is visible before Phase 2's paid APIs are added.

Auth and Persistence

Open decision the original design left unresolved — recommendation below, flagged for confirmation:

- **v1 (recommended)**: anonymous, session-scoped trips (a `userId` of `null` in the Trip State Model), stored server-side keyed by a session/browser id so refresh doesn't lose state, but not tied to an account. No auth system to build for the demo.

- **v2**: add accounts (e.g. NextAuth) once there's a reason to persist trips across devices or make them shareable — this changes the backend to require a `users` table and auth middleware on the API routes, and turns `userId` from always-null into a real foreign key.

Stack Decision

Vercel AI SDK (`ai` package), not LangChain. Rationale: this is already a Next.js App Router project, and the AI SDK is built specifically for that combination — native streaming helpers, typed tool calling, and route-handler integration — without LangChain's extra abstraction layer, which mainly earns its keep in more complex multi-agent or multi-vector-store systems this project doesn't need.

This still holds with MCP in the mix: the AI SDK has a built-in MCP client (`experimental_createMCPClient`) that connects to an MCP server and exposes its tools to the agent loop exactly like a locally-defined tool — so adopting MCP servers for Google Maps and Amadeus doesn't change the stack decision, it's additive within it, not a reason to reconsider LangChain.